DESARROLLO WEB CLIENTE - 2º DAW

UD 02: Javascript y React

EJEMPLO DE AUTOAPRENDIZAJE EA12

Arrays con callbacks: forEach, map, filter, every, some y reduce

Curso 2026-2027

Elaborado por: José Fernández González

Versión 1.1 · septiembre de 2026

CC BY-NC-ND 4.0

https://github.com/jstechg/material-docente/tree/main/2DAW_Desarrollo_Web_Cliente

Semana	Hora	Tiempo	Modalidad
2	1	20 min	Guiado en clase

1. Qué se aprende

- Diferenciar métodos de recorrido y transformación.
- Comprender qué métodos devuelven un nuevo array y cuáles devuelven boolean o un valor acumulado.
- Consolidar callbacks y funciones flecha.

2. Enunciado del ejemplo

Partiendo de un array de consumos eléctricos, mostrar valores, transformarlos, filtrar los altos y obtener comprobaciones y total acumulado.

3. Ficheros necesarios

Crea exactamente la estructura indicada y copia el contenido de cada fichero. Los ejemplos están redactados específicamente para este material de aula.

index.html

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EA12 - Callbacks de Array</title>
</head>
<body>
  <h1>EA12 - Callbacks de Array</h1>
  <p>Abre las herramientas de desarrollo con F12 y consulta la consola.</p>

  <script src="app.js"></script>
</body>
</html>
```

app.js

```
"use strict";

const consumos = [120, 95, 180, 60, 140];

consumos.forEach((valor, indice) => {
  console.log(`Lectura ${indice}: ${valor} W`);
});

const enKilovatios = consumos.map((valor) => valor / 1000);
const altos = consumos.filter((valor) => valor >= 130);
const todosPositivos = consumos.every((valor) => valor > 0);
const existeBajo = consumos.some((valor) => valor < 80);
const total = consumos.reduce((acumulador, valor) => acumulador + valor, 0);

console.log(enKilovatios);
console.log(altos);
console.log(todosPositivos);
console.log(existeBajo);
console.log(`Total: ${total} W`);
```

4. Preparación y ejecución paso a paso

1. Crear EA12_array_callbacks.
2. Ejecutar y relacionar cada salida con el método que la genera.
3. Cambiar un consumo a -5 y comprobar every().
4. Cambiar el umbral de filter() y observar el nuevo array.

5. Resultado esperado / comprobaciones

- map devuelve un nuevo array de kW.
- filter devuelve [180,140].
- every da true con los datos iniciales; some da true; reduce suma 595 W.

6. Comentarios didácticos

- Este ejemplo es un puente directo hacia React, donde map() se utilizará para generar elementos JSX sin modificar directamente el array original.